

NEXYGEN

Net-Zero Emissions Forecasting Platform

Technical Documentation & Project Report

Prepared for Nexygen Energy Ltd.

Author: Stephen Ogodu • Repository: github.com/stephenogodo/Nexygen-Net-Zero-Project

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	3
Headline Finding.....	3
What Was Delivered.....	3
2. Business Context.....	3
The Business Problem.....	4
Project Objectives.....	4
3. Original Scope & Delivery Status.....	4
3.1 Project Scope vs. Delivery.....	4
3.2 Day-by-Day Delivery Plan Reconciliation.....	5
4. Data Overview.....	5
Data Quality Finding.....	5
5. Methodology.....	6
5.1 Data Preparation Pipeline (data_prep_pipeline.py).....	6
5.2 Exploratory Data Analysis Pipeline (eda_report.py).....	6
Anomaly detection: local baseline, not raw z-score.....	6
Anomaly threshold: calibrated to the data, not assumed.....	6
Seasonality: reported as amplitude, not just peak month.....	6
5.3 Training Pipeline (train_pipeline.py).....	6
6. Key Findings.....	7
6.1 Net-Zero Target Gap (Primary Finding).....	7
6.2 Emissions Trend & Seasonality.....	7
6.3 Anomaly Detection.....	8
6.4 Emissions Drivers.....	8
6.5 Model Selection: SARIMA vs. Prophet.....	8
7. System Architecture.....	9
7.1 Backend API.....	9
7.2 Frontend Dashboard.....	10
7.3 Deployment.....	10
8. Engineering Practices & Quality Assurance.....	10
8.1 Automated Testing & CI/CD.....	10
8.2 Issues Discovered and Resolved.....	11
9. Known Limitations & Future Work.....	12
10. Conclusion.....	12
Appendix A: Repository Structure.....	12
Appendix B: Key Commands.....	13
Run the project.....	13
Retrain models.....	13
Regenerate the EDA report.....	13
Run tests.....	13
Smoke-test a running deployment.....	14

1. Executive Summary

This report documents the design, build, and validation of the NEXYGEN Net-Zero Emissions Forecasting Platform — a full-stack, containerized system that forecasts Scope 1 and Scope 2 carbon emissions for Nexygen Energy Ltd., a UK energy provider publicly committed to net-zero operational emissions by 2040.

The platform moves the business from historical emissions reporting to forward-looking forecasting: it automatically selects the better-performing model (SARIMA or Prophet) per emission scope, serves forecasts and scenario analysis through a REST API, and surfaces the results through an interactive dashboard — including a target-gap view that tracks actual progress against the company's own published reduction path.

Headline Finding

Nexygen Energy is behind its own published net-zero reduction path in every year since the 2020 baseline.

By 2025, actual emissions reduction stands at 22.0% versus a targeted 24.64% — a gap of 2.64 percentage points. The gap has grown from 1.11 points in 2021 to 2.64 points in 2025, with a brief narrowing in 2022 before resuming its widening trend through 2025.

What Was Delivered

- A data validation, EDA, and model-training pipeline replacing an original single exploratory notebook
- An automated model-selection system that fits both SARIMA and Prophet per emission scope and ships only the higher-accuracy candidate
- A FastAPI backend exposing 6 endpoints (forecast, scenario, historical data, target-gap, model audit, health) with opt-in authentication
- A Streamlit dashboard with four interactive views: Forecast, Scenario, Historical Data, and Target Gap
- A CI/CD pipeline (GitHub Actions) that runs automated tests, re-executes the training pipeline end-to-end, and builds — and boots — the real Docker images on every push
- 27 automated test cases covering the API surface, model selection logic, and authentication states
- A single cross-platform entry point (run.py) supporting both local and Docker execution

2. Business Context

Nexygen Energy Ltd. is a mid-to-large UK electricity and gas provider, founded in 2009, serving over 1.5 million customers across England, Scotland, and Wales. Over the past decade the company has moved from a competitive-pricing supplier toward a sustainability-driven organization, expanding into renewable energy sourcing and smart-meter analytics, and publicly committing to net-zero operational emissions by 2040.

The Business Problem

As Nexygen's renewable portfolio and regulatory/investor scrutiny grew, the company faced a capability gap: it had robust systems for tracking historical emissions but no reliable way to forecast future Scope 1 (direct: on-site fuel combustion, vehicles) or Scope 2 (indirect: purchased electricity) emissions. This made it impossible to answer basic strategic questions:

- Are current reduction initiatives sufficient to meet the 2040 net-zero commitment?
- What does the emissions trajectory look like under different operational scenarios?
- Where, specifically, is the company falling short of its own published targets?

Contributing factors included tightening UK/EU regulatory disclosure requirements, investor demand for forward-looking ESG data, and operational complexity from fluctuating energy demand and fuel mix. The business risk was concrete: regulatory non-compliance, reputational exposure, and misallocated capital on reduction initiatives that were not being measured against forecast trajectories.

Project Objectives

- Centralize and standardize Scope 1/2 emissions data into a single analytical pipeline
- Develop accurate short-, medium-, and long-term emissions forecasts
- Compare projected emissions against the company's published reduction pathway to identify gaps
- Enable scenario analysis for proposed efficiency and renewable-sourcing initiatives
- Deliver the results through an accessible dashboard rather than a static report

3. Original Scope & Delivery Status

The project was commissioned against a defined scope (data ingestion, EDA, forecasting, API development, and containerization/deployment) and, separately, a day-by-day delivery plan. Both are reconciled against final delivery status below.

3.1 Project Scope vs. Delivery

Scope Item	Status	Delivered As
Data ingestion & validation	Complete	pipeline/data_prep_pipeline.py
Exploratory data analysis (trend, seasonality, anomalies)	Complete	pipeline/eda_report.py
Driver analysis	Complete	pipeline/eda_report.py (Random Forest)
Time-series forecasting (baseline + scenario)	Complete	pipeline/train_pipeline.py, /forecast, /scenario
Model accuracy evaluation	Complete	R ² /MSE/MAE, both candidates recorded

Scope Item	Status	Delivered As
API development (FastAPI)	Complete	app.py — 6 endpoints
Authentication & logging	Complete	Opt-in API-key auth; structured logging
Automated tests & health checks	Complete	27 pytest cases; /health endpoint
Containerization (Docker)	Complete	Dockerfiles + docker-compose.yml
Deployment documentation / runbook	Complete	RUNBOOK.md
Documentation and presentation	Complete	README.md, this report, presentation deck

3.2 Day-by-Day Delivery Plan Reconciliation

A separate day-by-day delivery plan was reconciled against the codebase mid-project. Two genuine gaps were identified and subsequently closed:

- EDA: emissions trend and anomaly analysis — originally present only in the retired exploratory notebook; rebuilt as pipeline/eda_report.py
- EDA: driver analysis — not previously implemented at all; added as a Random Forest feature-importance study within the same script
- Documentation and Presentation — documentation existed (README); a presentation deck did not. Both a technical/executive slide deck and a LinkedIn-format carousel were subsequently produced.

4. Data Overview

The platform is built on ESG_Data.csv — 236,736 rows of daily-granularity operational data spanning 2020-01-01 to 2025-12-31, covering 18 assets (A001–A018) across 5 asset types, 3 UK regions, and 3 energy types (electricity, gas, diesel), split into Scope 1 and Scope 2 emission records. The dataset also carries the company's published annual reduction path: Target_Emissions_tCO2e and Reduction_Percentage_vs_BaseYear, representing a 5.5%/year target reduction rate from the 2020 baseline.

Data Quality Finding

Scale mismatch in the published target column.

Target_Emissions_tCO2e is on a different absolute scale than the summed per-asset actuals (roughly 3x off), meaning the two cannot be compared directly as raw values. This is treated as a source-data characteristic, not a platform defect. The target-gap analysis therefore compares percentage reduction versus the 2020 baseline on both sides — a scale-independent, directly comparable metric — rather than the mismatched absolute figures.

5. Methodology

The original project used a single exploratory notebook. This was replaced with three focused, independently runnable, CI-tested pipeline scripts — chosen over a notebook specifically for reproducibility, diffability, and automated testability.

5.1 Data Preparation Pipeline (`data_prep_pipeline.py`)

Loads and validates the raw CSV (required-column checks, null/duplicate counts), aggregates it into monthly totals per emission scope, and splits each series into train/test at the 80th-percentile date. This produces a 58-month training window and a 14-month held-out test window, with the training window ending 2024-10-19 — the anchor date all API forecasts are generated from.

5.2 Exploratory Data Analysis Pipeline (`eda_report.py`)

Produces trend, seasonality, anomaly, and driver analysis as saved chart images and a findings.md summary. Two methodology decisions were made deliberately, after the naive default approach was found to produce misleading results on this dataset:

Anomaly detection: local baseline, not raw z-score

Scope 2 has a strong multi-year downward trend. A raw z-score computed against the whole series' mean flags high values from early in the series — before the trend declined — as “anomalies,” which is trend drift, not a genuine local anomaly. The pipeline instead computes a 30-day rolling-median local baseline and flags residuals against that baseline, isolating days that are unusual relative to their own recent context.

Anomaly threshold: calibrated to the data, not assumed

A conventional $z = 3.0$ threshold was tested first and found the maximum observed $|z|$ in this dataset is approximately 2.4 — meaning the conventional threshold would report zero anomalies regardless of the data's actual behavior. The threshold was recalibrated to $z = 2.0$ against the dataset's own distribution, surfacing a genuinely informative set of 133 candidate anomalous days rather than a vacuous null result.

Seasonality: reported as amplitude, not just peak month

Both Scope 1 and Scope 2 numerically peak in the same calendar month (June), which would suggest equally strong seasonality if only the peak month were reported. The pipeline instead reports peak-to-trough swing as a percentage of the mean — revealing that Scope 1 is genuinely, strongly seasonal (18.1% swing) while Scope 2 is essentially flat (1.4% swing) despite sharing a peak month.

5.3 Training Pipeline (`train_pipeline.py`)

For each emission scope, two candidate models are fit on the identical 58-month training split and evaluated on the identical 14-month held-out test split:

- SARIMA/SARIMAX (statsmodels) — order found via an `auto_arima` search
- Prophet (Meta/Facebook) — hyperparameters found via a grid search over `changepoint_prior_scale` and `seasonality_mode`

The candidate with the higher test-set R^2 is refit and shipped; both candidates' full metrics are recorded in `models/model_manifest.json` regardless of outcome, so the comparison remains auditable rather than simply asserted. Model selection is genuinely data-driven: a future retrain on different data could flip the winner, and the API would serve whichever model won without any code changes, via a model-registry abstraction (`model_registry.py`) that exposes a uniform `.forecast(steps)` interface over either model type.

SARIMA and Prophet were serialized differently by design: SARIMA via statsmodels' native pickle format, Prophet via its own JSON serialization — Prophet's maintainers do not guarantee pickle stability across its Stan-backend versions.

6. Key Findings

6.1 Net-Zero Target Gap (Primary Finding)

Comparing actual year-over-year percentage reduction in total (Scope 1 + Scope 2) emissions against the company's published target reduction path, both measured against the 2020 baseline:

Year	Actual Emissions (tCO ₂ e)	Actual Reduction	Target Reduction	Gap (pts)	Status
2020	45,013.0	0.00%	0.00%	0.00	Baseline
2021	43,038.0	4.39%	5.50%	1.11	Behind
2022	40,502.0	10.02%	10.70%	0.68	Behind
2023	38,776.7	13.85%	15.61%	1.76	Behind
2024	37,055.6	17.68%	20.25%	2.57	Behind
2025	35,108.8	22.00%	24.64%	2.64	Behind

The company has been behind its own published reduction path every year since the 2020 baseline. The gap is not strictly monotonic — it narrows from 1.11 points (2021) to 0.68 points (2022) before widening consistently from 2022 through 2025, reaching 2.64 points. Net effect over the full period: the shortfall has more than doubled.

This finding is precisely the kind of signal the platform was commissioned to surface: not a data anomaly to be explained away, but the primary output the business case for this project rests on.

6.2 Emissions Trend & Seasonality

Metric	Scope 1	Scope 2
Change, 2020→2025	-6.3%	-37.6%

Metric	Scope 1	Scope 2
Seasonality (peak-to-trough swing)	18.1% of mean — strong	1.4% of mean — negligible
Peak / trough month	June / December	June / December (nominal only)

Scope 2's steeper multi-year decline is the more visible trend, but the target-gap analysis (\$6.1) shows that decline is still not steep enough to meet the published target. The two scopes share a nominal peak month but are not comparably seasonal — Scope 1's swing is more than 12x the size of Scope 2's.

6.3 Anomaly Detection

Using the rolling-baseline residual method described in §5.2, 133 candidate anomalous days were identified across the full 2020–2025 period: 58 in Scope 1, 75 in Scope 2, each flagged where the residual against a 30-day local baseline exceeds $|z| = 2.0$. Anomalies are distributed across the full timeline rather than clustering at any single point, consistent with the local-baseline method correctly separating short-term irregularities from the underlying multi-year trend.

6.4 Emissions Drivers

A Random Forest feature-importance analysis was run across four categorical drivers (Asset Type, Location, Emission Type, Energy Type) to determine which factors most explain variation in per-record emissions:

Driver	Relative Importance
Asset Type	58.8%
Location	24.9%
Emission Type (Scope 1 vs. 2)	16.2%
Energy Type	0.0%

Asset Type alone accounts for more explanatory power than Location and Emission Type combined, with ControlRoom recording the highest mean emissions per row of any asset type. Operational_Status was excluded from this analysis entirely: it takes only the single value “Active” throughout the dataset, has zero variance, and therefore cannot explain any variation regardless of what a naively-included model coefficient might suggest.

6.5 Model Selection: SARIMA vs. Prophet

Both candidates were fit on the identical 58-month training window and scored on the identical 14-month held-out test window — an apples-to-apples comparison, not a default-settings fit for either side.

Scope	Model	Order / Params	R ²	MSE	MAE	Result
Scope 1	SARIMA	(3,1,2)(2,1,0,12)	0.964	566.3	20.6	
Scope 1	Prophet	cps=0.1, additive	0.982	286.2	14.9	WINNER
Scope 2	SARIMA	(0,1,1)(1,0,0,12)	0.792	523.2	16.5	
Scope 2	Prophet	cps=0.05, multiplicative	0.908	231.8	9.2	WINNER

Prophet won on both scopes — clearly on Scope 1 (R^2 0.964→0.982), decisively on Scope 2, where MAE nearly halved (16.5→9.2 tCO₂e) and R^2 improved from 0.792 to 0.908. Scope 2 is also the scope that previously shipped with a broken model (see §8.2, Issue #1) — this result confirms that fix produced a genuinely well-fit model, not merely a non-crashing one.

7. System Architecture

Data → pipeline scripts → model registry → API → dashboard. Both the API and the dashboard also run directly as local processes (no Docker) via a single entry point, `run.py`.

7.1 Backend API

Built with FastAPI. Six endpoints:

Method	Path	Auth	Purpose
GET	/health	Public	Service status, active model + R^2 per scope, auth state
POST	/forecast	Protected*	N-month forecast for a given scope
POST	/scenario	Protected*	Forecast with an adjustable annual reduction-rate assumption
GET	/historical	Protected*	Historical emissions, filterable by scope/granularity/date/asset/location
GET	/target-gap	Protected*	Actual vs. target % reduction, per year, with on-track verdict
GET	/model-info	Protected*	Full training manifest: winning model + both candidates' metrics, per scope

* Protected endpoints require a matching X-API-Key header only if the `API_KEY` environment variable is set; if unset, the API is fully open. This is a deliberate, opt-in design intended for a small internal deployment (a single shared key, no per-user roles) rather than a full identity system — documented as a known limitation, not hidden. Swagger UI documentation is auto-generated at `/docs`.

7.2 Frontend Dashboard

Built with Streamlit. Four interactive views, each consuming a live backend endpoint: Forecast, Scenario (what-if analysis with an adjustable reduction-rate slider), Historical Data (filterable trend charts), and Target Gap (the primary finding, §6.1, rendered as an actual-vs-target chart with an on-track/behind-target indicator). A sidebar displays the active model and R^2 per scope, pulled live from /model-info.

7.3 Deployment

- Docker + Compose: separate backend/frontend images; frontend startup is health-gated on the backend via depends_on: service_healthy
- .env-driven configuration: ports, model directory, log level, API_KEY — no hardcoded values; .env.example documents every setting
- Health checks: Docker HEALTHCHECK directives plus Compose's service_healthy condition, not just a bare liveness ping
- run.py: single cross-platform entry point — `python run.py --env {local,docker} --app {fastapi,streamlit,both}`

8. Engineering Practices & Quality Assurance

8.1 Automated Testing & CI/CD

27 pytest cases cover the API surface end-to-end: health and forecast endpoints (including input-validation boundary cases), the scenario endpoint's zero-rate-matches-baseline invariant, historical data filtering, target-gap analysis (single year, all years, unknown year), both open and enforced authentication states, and model-selection integrity — asserting that the model reported as active by /health and /model-info is verifiably the higher- R^2 candidate from the training manifest, not merely present.

A three-job GitHub Actions CI pipeline runs on every push:

- backend-tests — the full pytest suite
- pipeline-smoke-test — re-runs data preparation, EDA, and training end-to-end and asserts a valid model was selected per scope, independent of whatever models happen to be committed
- docker-build — builds both Docker images and boots the backend container, polling /health until it reports healthy

The docker-build job specifically closes a real gap: it would have caught the dockerfile-casing defect (Issue #2 below) automatically, on every push, rather than relying on manual discovery.

8.2 Issues Discovered and Resolved

This project surfaced a substantial number of real defects during development — in the original codebase, and introduced by tooling used during the build itself. They are logged here in full rather than omitted, because the debugging and correction process is itself part of the engineering record.

#	Issue	Root Cause	Resolution
1	Scope 2 model shipped broken ($R^2 = -1.96$, never converged)	auto_arma order was tuned on daily data, then wrongly applied to monthly data — severely overparameterized for ~58 monthly points	Retrained with a proper monthly-series order search; later fully superseded by the SARIMA-vs-Prophet pipeline (§5.3)
2	docker compose up --build silently failed	docker-compose.yml referenced dockerfile: Dockerfile (capital D); actual filename was lowercase dockerfile — fails on case-sensitive filesystems	Corrected the reference; added a CI job that builds the real image on every push so this class of defect can't recur silently
3	Naive anomaly detection produced misleading results	Raw z-score against the whole series mistook multi-year trend drift for local anomalies	Replaced with a 30-day rolling-baseline residual method (§5.2)
4	Anomaly threshold reported zero findings	Conventional $z = 3.0$ exceeds this dataset's actual maximum observed $ z $ (~2.4)	Recalibrated to $z = 2.0$ against the dataset's own distribution
5	frontend/requirements.txt failed to install on modern Python	Hard-pinned numpy==1.26.0 (unsupported on Python ≥ 3.13) and other stale, unnecessary pins predating this project's work	Rewrote to pin only actually-imported packages, unversioned
6	Local .env file silently ignored by the frontend	frontend/.env.example existed, but streamlit_app.py never called load_dotenv()	Added the missing call; verified a local .env's values are actually picked up
7	run.py local mode: frontend could not reach the backend	Frontend's default API_BASE_URL (http://backend:8000) only resolves inside Docker's network; local mode never overrode it	Injected a localhost-appropriate default into the subprocess environment, without overriding an explicitly-set value
8	Repository structure used a misleading top-level folder name	docker_practice/ predated this project and no longer reflected its contents (the entire deployable application)	Flattened the structure; backend/, frontend/, and docker-compose.yml moved to repository root
9	Stale path references survived the restructure	Two .env.example files and one planned git rm were missed when docker_practice/ was flattened	Located and corrected via full repository greps, not assumed complete
10	Docker Desktop pause left zombie network processes	Pausing (rather than stopping) Docker Desktop left com.docker.backend / wslrelay bound to the app's ports, causing	Diagnosed via netstat process-ID tracing; resolved via a full Docker Desktop quit and wsl --shutdown

#	Issue	Root Cause	Resolution
		non-deterministic request routing	

9. Known Limitations & Future Work

- No database — ESG_Data.csv is read directly at API startup. Adequate at the current data volume; worth revisiting if write-access or significantly larger data volumes become a requirement.
- No CI-driven auto-deploy — CI validates every push but does not deploy; cloud deployment (AWS/Azure/GCP) is a natural next step.
- SHAP-based interpretability is intentionally not implemented — it does not natively apply to SARIMA or Prophet, neither of which exposes the kind of per-feature attribution SHAP is built for. Adding it meaningfully would require a genuinely different, feature-based model (e.g., a gradient-boosted regressor on lag and seasonal features) rather than a superficial addition.
- Single shared API key, no per-user roles — adequate for a small internal deployment; multi-user access control would need a proper OAuth2/JWT scheme.
- Hyperparameter search is a small grid/auto-order search, not a full AutoML sweep — appropriate for two ~72-month monthly series; would need rethinking for a much larger model portfolio.
- Driver analysis is descriptive, not causal — Random Forest importances rank which categorical factors are associated with emissions variance; they are not a causal claim about why those factors matter.

10. Conclusion

The NEXYGEN platform delivers what the original business case required: a system that moves Nexygen Energy from reactive historical reporting to forward-looking, auditable emissions forecasting. The model-selection methodology is genuinely comparative rather than asserted, the primary finding (a widening gap against the company's own published net-zero path) is derived directly from the platform's own target-gap logic rather than pre-supposed, and the engineering practice around the build — automated testing, CI, and an honestly logged defect history — reflects production software discipline rather than a one-off analytical exercise.

The headline result is a concrete, actionable one for the business: on current trend, Nexygen Energy will not meet its 2040 net-zero commitment without either accelerating its reduction initiatives or revising its published targets. The platform's /scenario endpoint and dashboard tab exist specifically to let leadership test the former before committing budget to it.

Appendix A: Repository Structure

Nexygen-Net-Zero-Project/

```

|-- README.md, RUNBOOK.md, run.py, LICENSE, ESG_Data.csv
|-- .github/workflows/ci.yml
|-- presentation/
|   |-- NEXYGEN_Presentation.pptx
|-- docker-compose.yml
|-- .env.example
|-- backend/
|   |-- app.py                (FastAPI app)
|   |-- model_registry.py     (uniform SARIMA/Prophet interface)
|   |-- pipeline/
|       |-- data_prep_pipeline.py
|       |-- eda_report.py
|       |-- train_pipeline.py
|   |-- data/ESG_Data.csv
|   |-- reports/              (EDA output: 4 PNGs + findings.md)
|   |-- models/
|       |-- model_manifest.json
|       |-- scopel_model.json  (Prophet, winner)
|       |-- scope2_model.json  (Prophet, winner)
|   |-- tests/test_api.py     (27 pytest cases)
|   |-- dockerfile
|   |-- requirements.txt / requirements-train.txt / requirements-dev.txt
|-- frontend/
|   |-- streamlit_app.py
|   |-- dockerfile.streamlit
|   |-- requirements.txt

```

Appendix B: Key Commands

Run the project

```

python run.py --env local --app both          # local, both services
python run.py --env docker --app both         # Docker Compose, both
services

```

Retrain models

```

python pipeline/data_prep_pipeline.py --input data/ESG_Data.csv --output
data/processed

python pipeline/train_pipeline.py --input data/ESG_Data.csv --output
models

```

Regenerate the EDA report

```

python pipeline/eda_report.py --input data/ESG_Data.csv --output reports

```

Run tests

```

cd backend && pip install -r requirements-dev.txt && pytest tests/ -v

```

Smoke-test a running deployment

```
curl http://localhost:8000/health
```

```
curl -X POST http://localhost:8000/forecast -H "Content-Type: application/json" -d '{"emission_type": "scope1", "steps": 6}'
```